

# Podpora počas behu

## Runtime Environments

Ján Šturc, Richard Ostertág,  
Jana Kostičová

# Zahrňuje:

- Správu pamäti
- Volanie funkcií a procedúr
- Adresáciu dátových štruktúr
- Štandardné (zabudované funkcie)
- Podporu pre tabuľku symbolov

# Abstract Machine

Defines language semantics

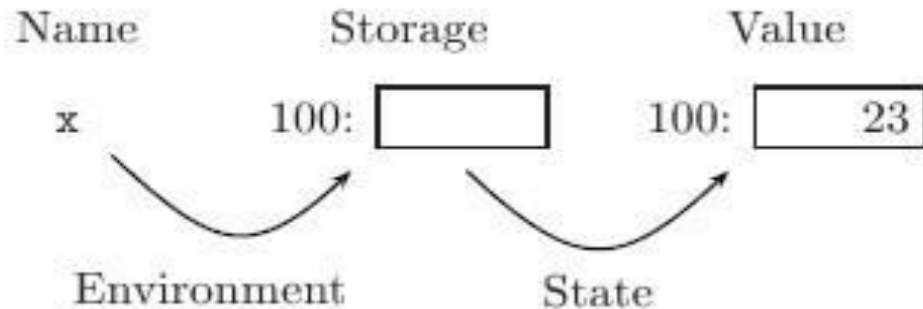
- How expressions are evaluated
- How statements execute
- How data is stored and manipulated
- Side effects, control flow, and function calls
- What behaviors are defined / undefined / implementation-defined

Compiler must preserve **observable behavior** defined by abstract machine.

Compiler (together with linker and runtime system) creates and manages run-time environment in which it assumes its target programs are executed.

## Binding of names

(Term “environment” within the programming language semantics)



An *environment* is a function that binds names to storage locations (lvalues) that are either absolute or relative to a pointer (e.g. the stack pointer). The environment changes when execution moves from one scope to another.

A *state* is a function that maps lvalues to rvalues. The state changes when the program executes an assignment statement.

The environment is controlled by the compiler, with help from the linker and runtime system. The state is controlled by the program.

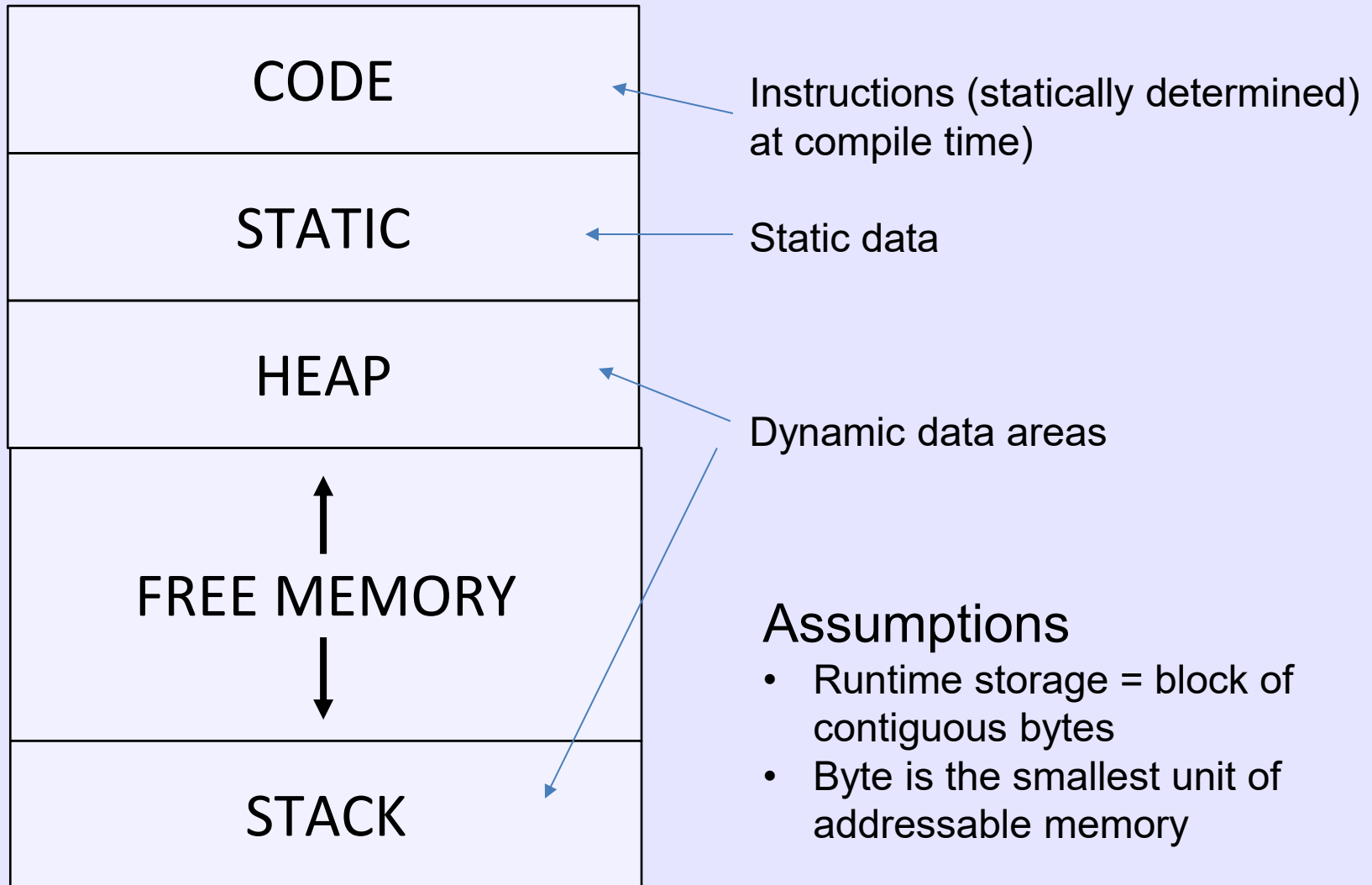
# Run-Time Environments

- Static vs. Runtime
- Mapping a high-level language to low-level machine environment implies generating code for allocating, maintaining and deallocating data objects to support this activity.
  - Allocation of storage locations
  - Access to variables and data
- Must implement abstract machine rules

# Procedures/Functions

- Control Abstraction
  - call/return semantics, parameters, recursion
- Controlled Namespace
  - Scope (local/non-local), binding, addressing
- External Interface
  - separate compilation, libraries (not dealing with here)

# Typical memory layout



# Static vs dynamic decisions

## Storage allocation decisions

### **Static**

- made at compile time

### **Dynamic**

- made at runtime (cannot be made at compile time)
  1. stack – procedure local variables
  2. heap – data that may outlive the procedure call



# Example

```
var a: array [0 .. 10] of integer;
```

```
procedure readarray ()
```

```
  var i: integer
```

```
  begin ... a[i] ... end
```

```
function partition(y,z: integer): integer
```

```
  var i,j,x,v: integer begin ... end
```

```
procedure quicksort(m,n: integer)
```

```
  var i: integer
```

```
  begin i := partition(m,n); quicksort(m,i-1); quicksort(i+1,n) end
```

```
procedure main()
```

```
  begin readarray(); a[0] = -9999; a[10] = 9999; quicksort(1,9); end
```

# Procedure Call Issues

- Control flow (call and return)
- Data: (formal/actual) parameters and return values
- Scope information
- Responsibilities of caller vs. callee
- Recursion
- Variable addressing

# Call Graphs

A **call graph** is a directed multi-graph where:

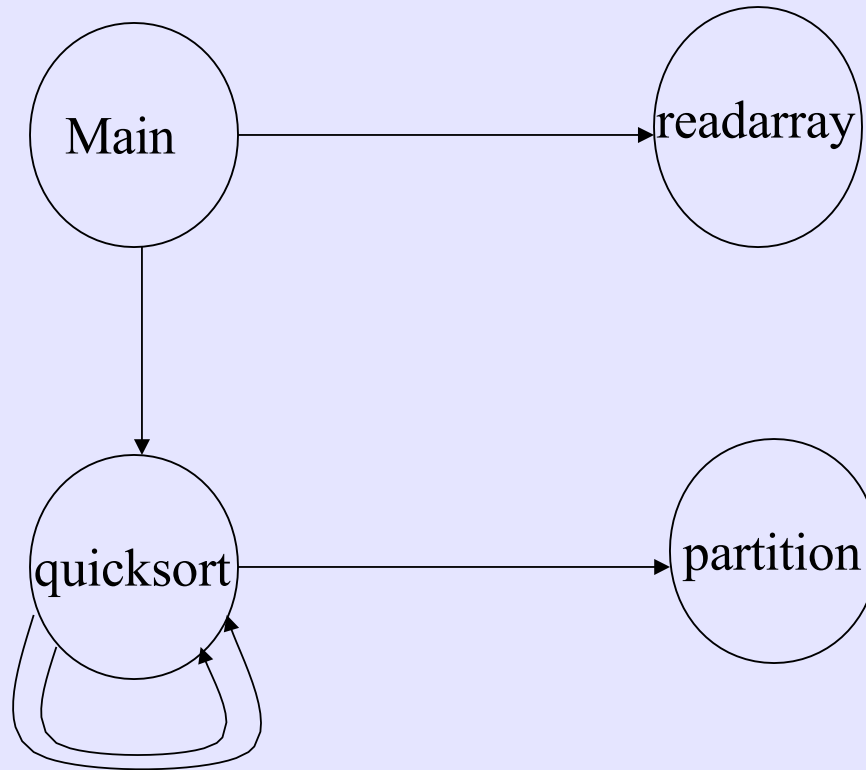
- the nodes are the procedures of the program and
- the edges represent calls between these procedures.

Used in optimization phase

Acyclic -> no recursion in the program

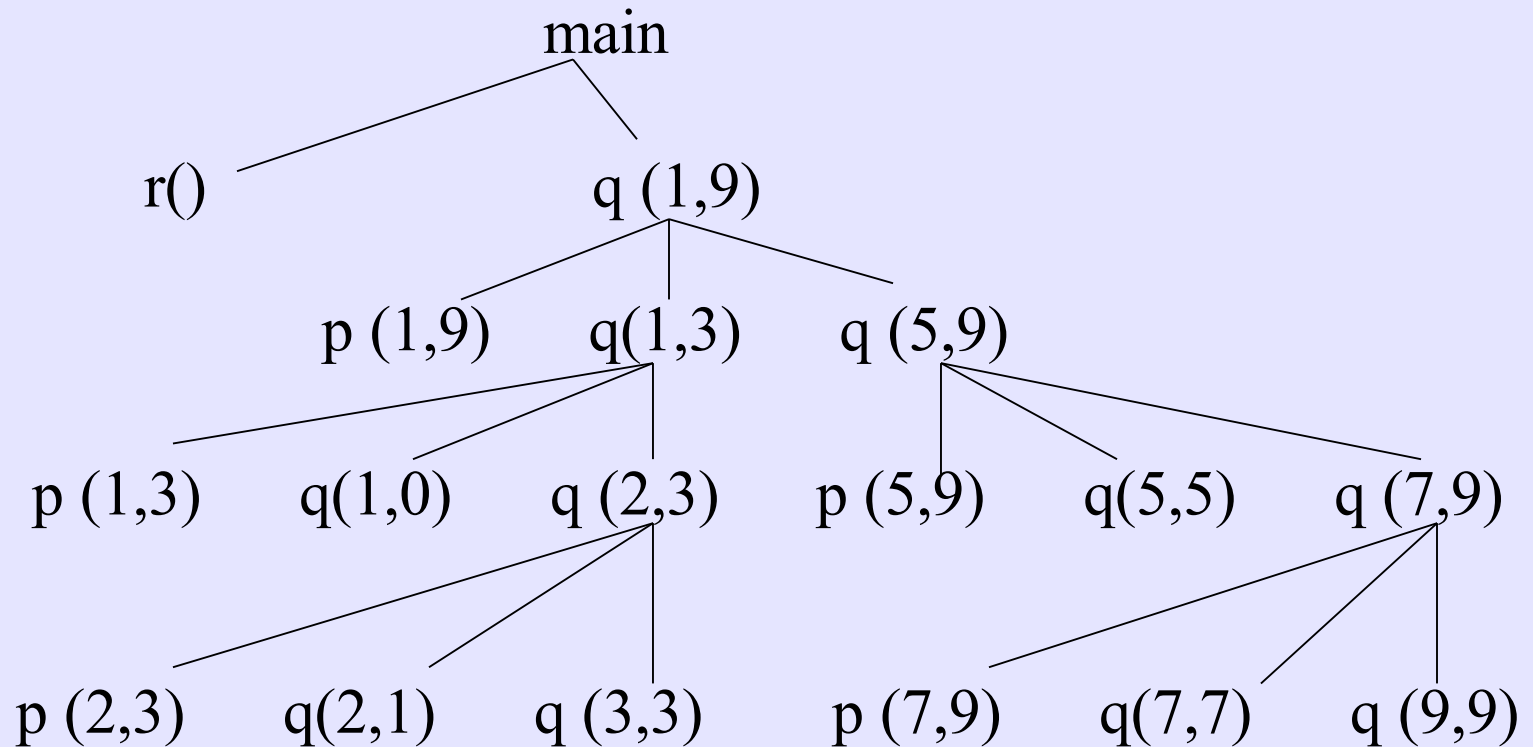
Can be computed **statically**.

# Call Graph for Example



# Run-time Control Flow

Call Tree - cannot be computed statically



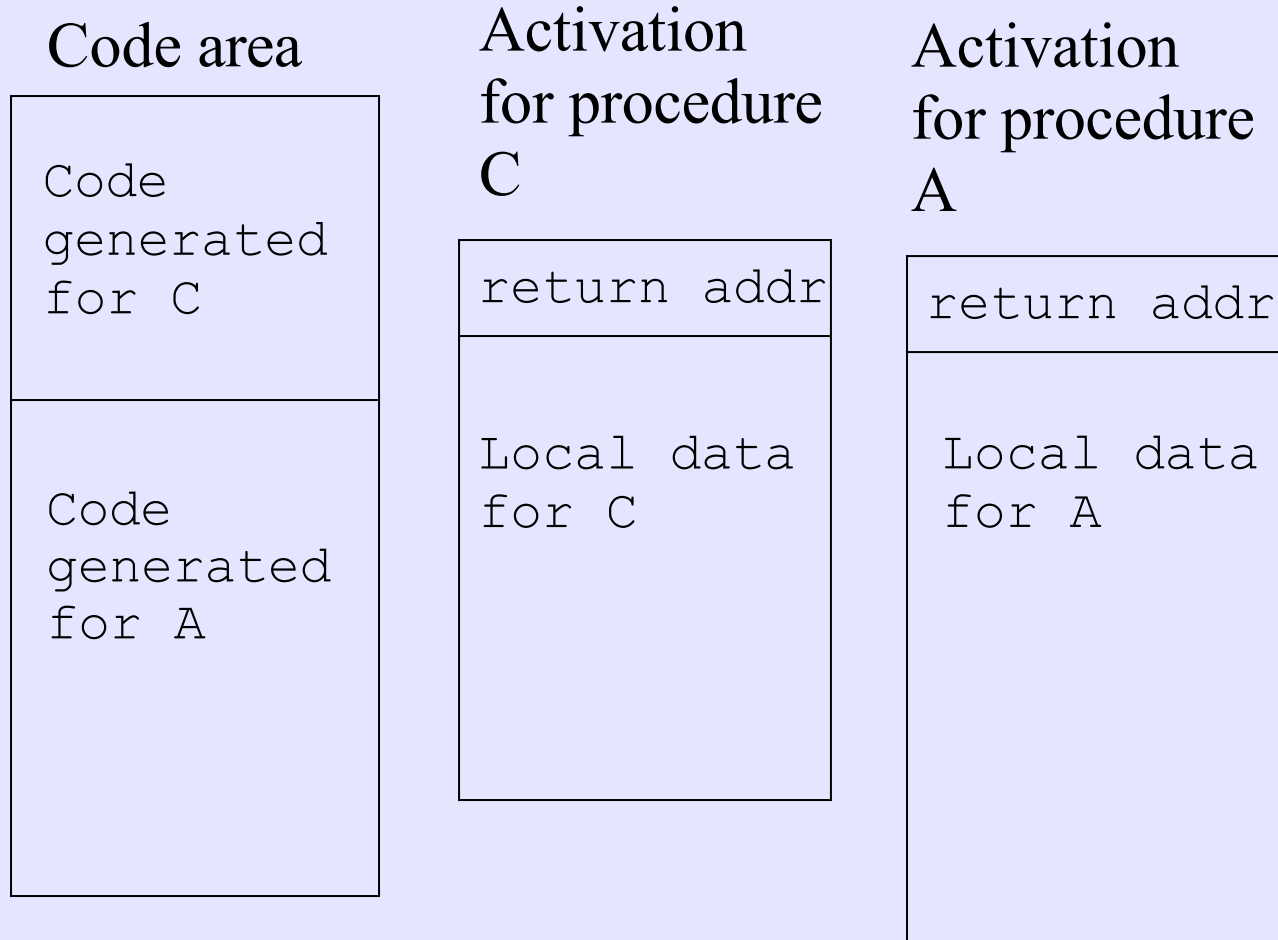
# Static Allocation

All space allocated at compile time.

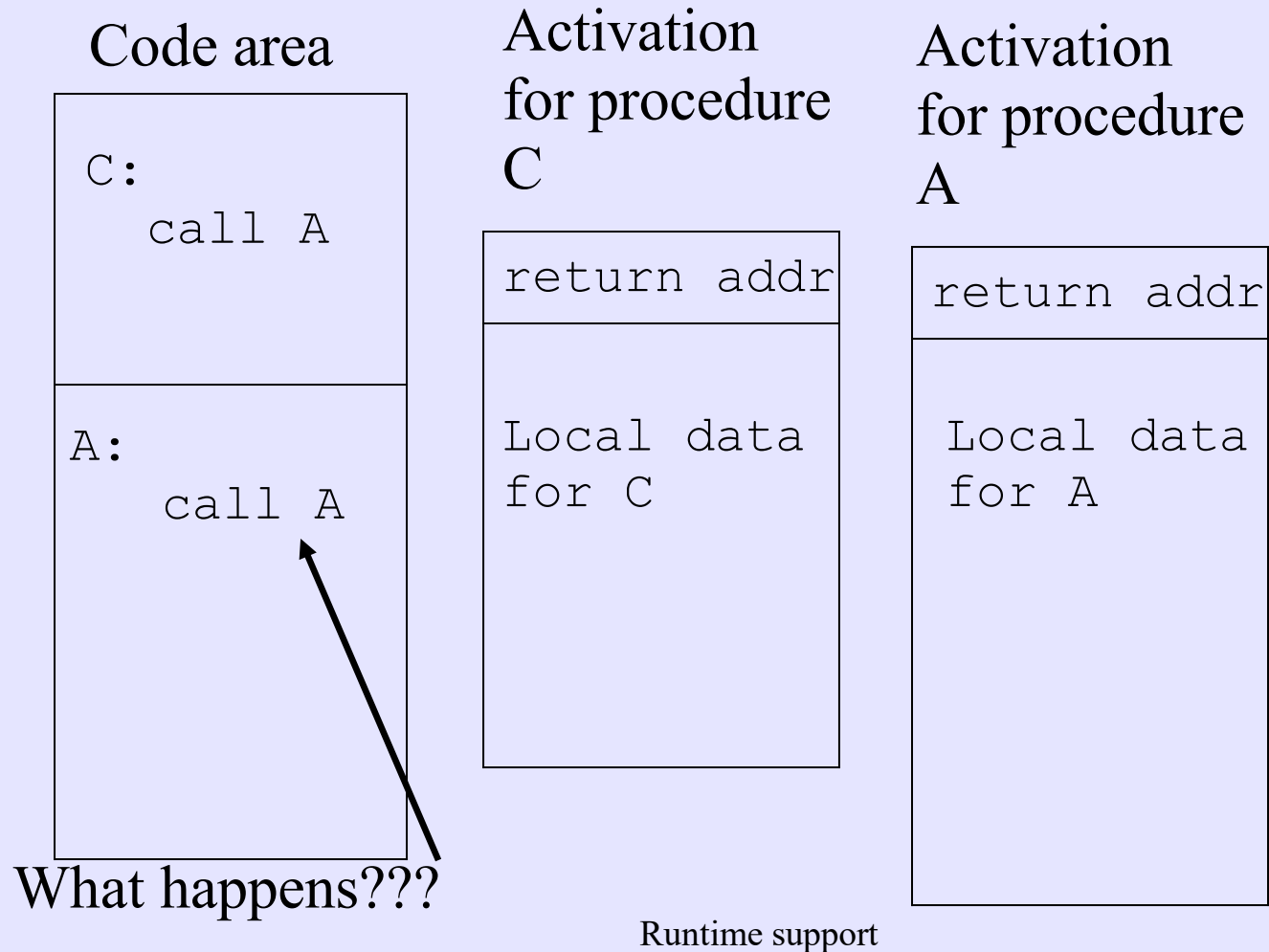
- Code area – machine instructions for each procedure
- Static area –
  - single data area allocated for each procedure.
    - local vars, parameters, return value
  - return address for each procedure.

No recursion

# Static Allocation



# Static Allocation





# Stack Allocation

- Code area – machine code for procedures
- Static data – not associated with procedures
- Stack – runtime information
  - Return addresses, scope information
- Activation records – allocated at call time onto a runtime stack.

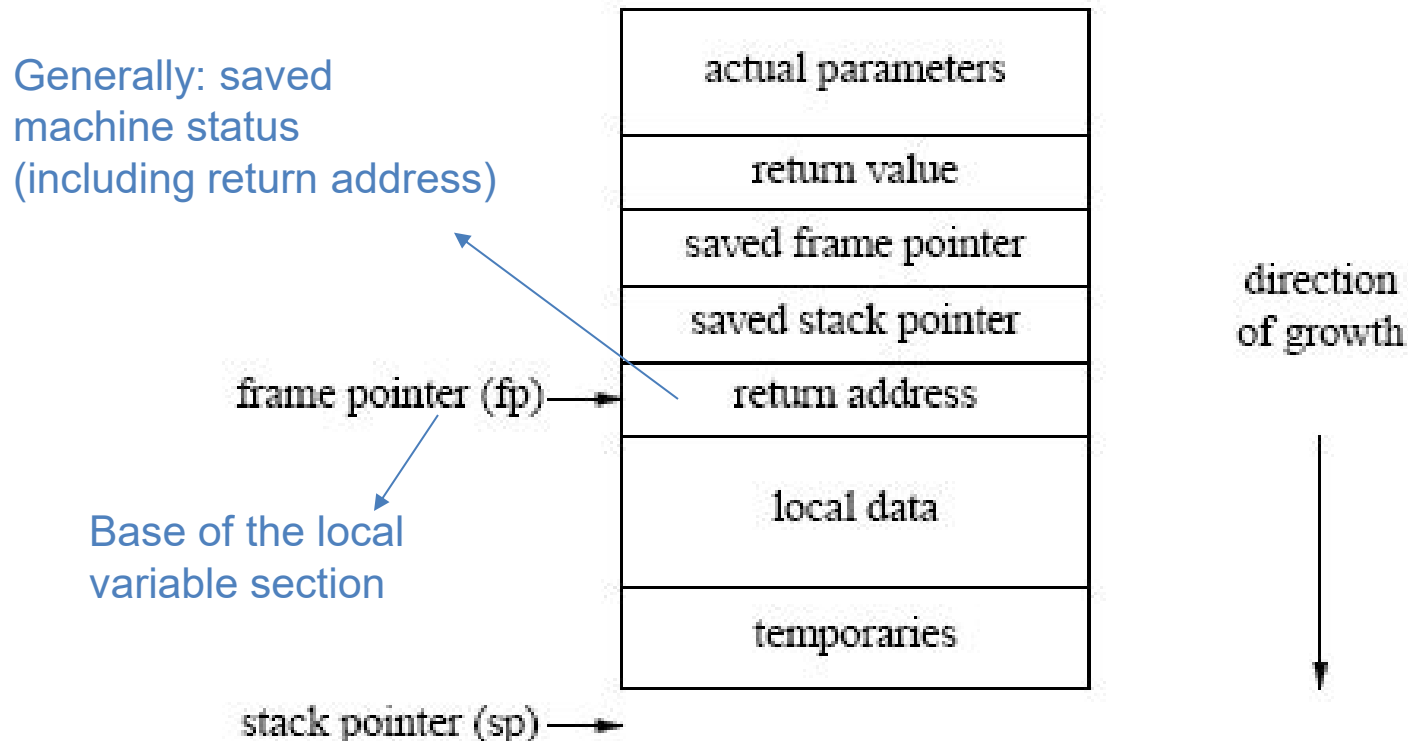
# Activation Records

Information needed by a single instance of a procedure.

- Local data
- Parameter storage
- Return value storage
- Control links for stack
- Return address

# Activation records

Most platforms have an Application Binary Interface (ABI) standard that dictates the form of ARs. They usually look similar to this:



# Activation Records (AR)

Different procedures/functions will have different size of activation records.

Activation record size can be generally determined at compile time.

Exceptions exist

- Variable-length arrays (unknown length)
- Variadic functions (unknown number of parameters)

# Stack Allocation - 1

# Call Tree

Diagram illustrating the relationship between **Main** and **readarray**:

```
graph TD; Main --- readarray;
```

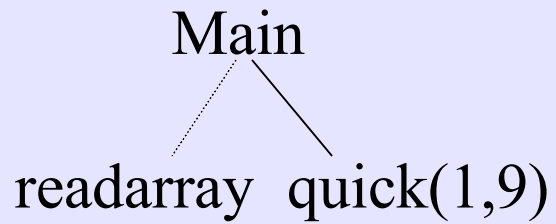
## Stack (growing downward)



## Runtime support

# Stack Allocation - 2

# Call Tree

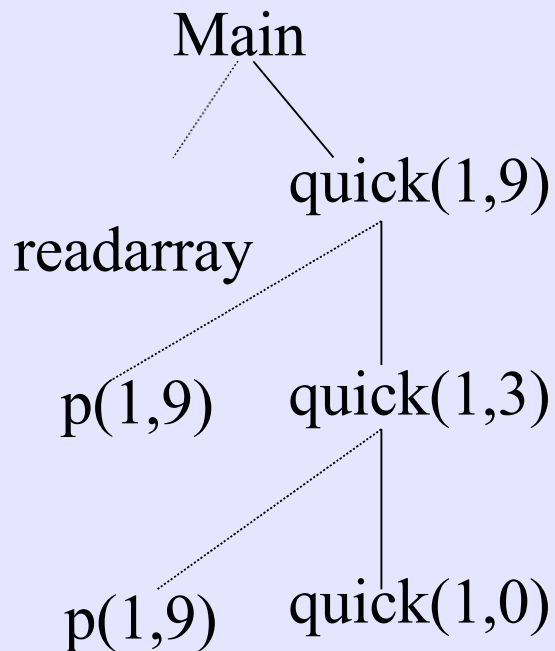


## Stack (growing downward)

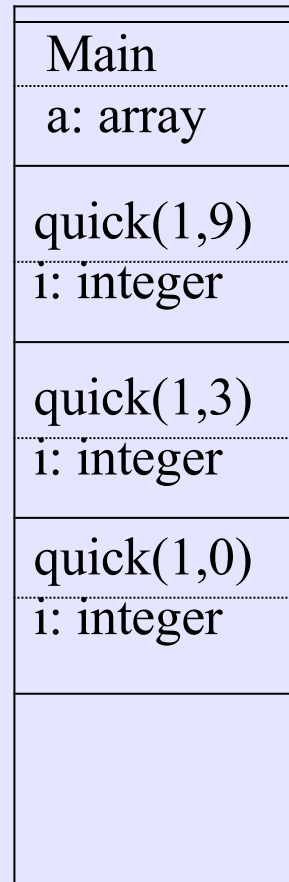
|                          |
|--------------------------|
| Main                     |
| a: array                 |
| quick(1,9)<br>i: integer |
|                          |

# Stack Allocation - 3

Call Tree



Stack (growing downward)



Runtime support

# Calling / return sequences

## **Calling sequence**

= code that allocates an activation record on the stack and enter information into its fields

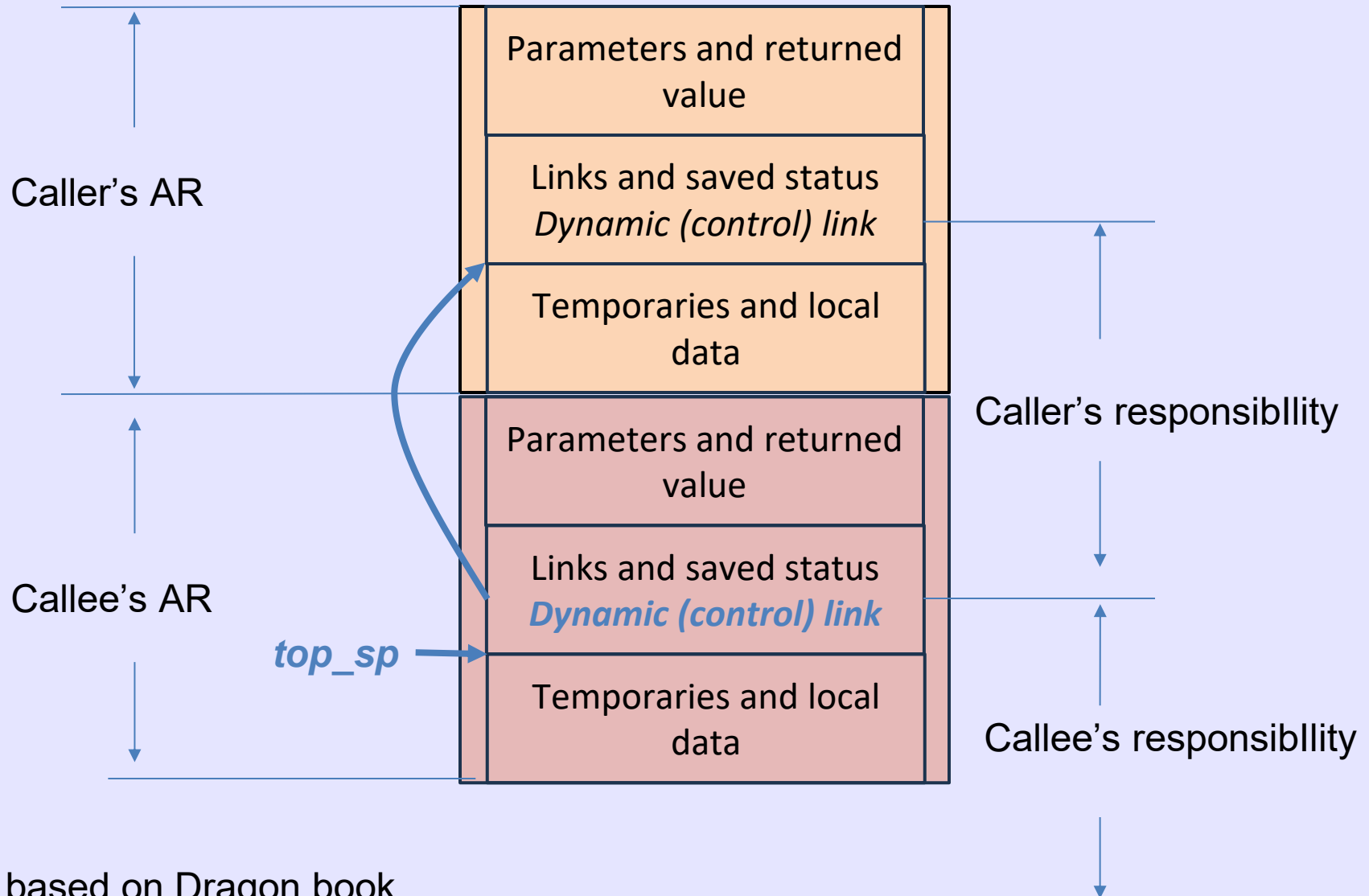
## **Return sequence**

= restores the state of the machine so that calling procedure can continue its execution after the call

Significant differences even among multiple implementations of the same languages



# Conceptual model of ARs\*



\* based on Dragon book

# Call Processing: Caller

- Create new (callee) activation record on stack.
- Evaluates actual parameters and places them in (callee) activation record.
  - Real ABIs:** Parameters are often passed (partially) via registers. Stack parameters are mostly left within the caller's AR.
- Stores a return address, dynamic link (= a link to caller activation record ***top\_sp*** ), and static link information into (callee) activation record
  - Real compilers:** *top\_sp* often corresponds to „frame pointer“ (FP). Callee saves the old FP and sets the new FP
- Make ***top\_sp*** to point at new (callee) activation record.
- Updates the program counter (***PC***) to the code area for the called procedure.

## Added at the point of the call

# Call Processing: Callee

- Save registers and other status data

**Real ABIs:** Some registers may be stored by caller, other by callee

- Initializes local data, including moving parameters (depends on calling convention)
- Begins local execution

**Added at the start of the function**

# Return Processing: Callee

- Place return value (if any) in activation.

**Real ABIs:** often stored in accumulator register

- Restore *top\_sp* and *PC*.

**Added at the ‘return’ point(s) of the function**

# Return Processing: Caller

- Restore registers and status
- Copy the return value (if any) from activation
- Continue local execution

**Added after the point of the call**

## Variable numbers of arguments

Some functions, like `printf`, take a variable number of arguments. Usually, there are fixed number (usually one or two) required arguments (e.g. the format string) that determine how many other arguments are expected.

Variable-length argument lists are easy to support if all arguments are passed on the stack; you simply access the argument list as an array. Support is much more complicated if some arguments are passed in registers, which is why many compiler writers wish variable-length argument lists would go away.

Supporting such functions imposes restrictions on ABI designers. For example, the ABI can't say that the caller pushes the arguments but the callee pops them (together with its own locals and temps), because in the presence of bugs, the two may disagree on how many arguments there are.

# Runtime Addressing

- Given a variable reference in the code, how can we find the correct instance of that variable?
- Tied to issues of scope

# Scope

The **scope** of a variable is that portion of the programs to which the variable applies.

- A variable is **local** to a procedure if the declaration occurs in that procedure.
- A variable is **non-local** to a procedure if it is not local to that procedure but the declaration occurs in an enclosing scope of that procedure.
- A variable is **global** if it occurs in the outermost scope.



# Types of Scoping

- Static – scope of a variable determined from the source code. Scope A is enclosed in scope B if A's source code is nested inside B's source code.
  - Data access is simpler in languages that do not allow nested procedure declarations (e.g., C family)
- Dynamic – current call tree determines the relevant declaration of a variable use.

# Most Closely Nested Rule

The scope of a particular declaration is given by the most closely nested rule

- The scope of a variable declared in block B, includes B.
- If x is not declared in block B, then an occurrence of x in B is in the scope of a declaration of x in some enclosing block A, such that A has a declaration of x and A is more closely nested around B than any other block with a declaration of x.

# Runtime Addressing in Static Allocation

- Variable addresses hard-coded, usually as offset from data area where variable is declared.
  - $\text{addr}(x) = \text{start of } x\text{'s local scope} + x\text{'s offset}$

# Control Links in Stack Allocation

- Dynamic link – points to caller's activation (old *top\_sp* pointer)
- Static (access) link – points to enclosing scope
  - If callee is directly enclosed by caller,  
*callee's static link = caller's activation*
  - If callee has the same enclosing scope as caller,  
*callee's static link = static link in caller's activation*
  - If callee is in a scope ( $k$  levels up) that encloses the caller, need to traverse  $k$  static links from the caller and use that static link.  
*callee's static link = caller's static link followed  $k$  times*

# Runtime Addressing in Stack Allocation

- At runtime, we can't know where the relevant activation record holding the variable **x** exists on the stack
- Use static (access) links to enable quick location
  - $\text{addr}(x) = \# \text{ static links} + x\text{'s offset}$
  - Local: (0,offset)
  - Immediately enclosing scope: (1,offset)

# Example Program

Program main;

a,b,c: real;

procedure sub1(a: real);

d: int;

procedure sub2(c: int);

d: real;

body of sub2

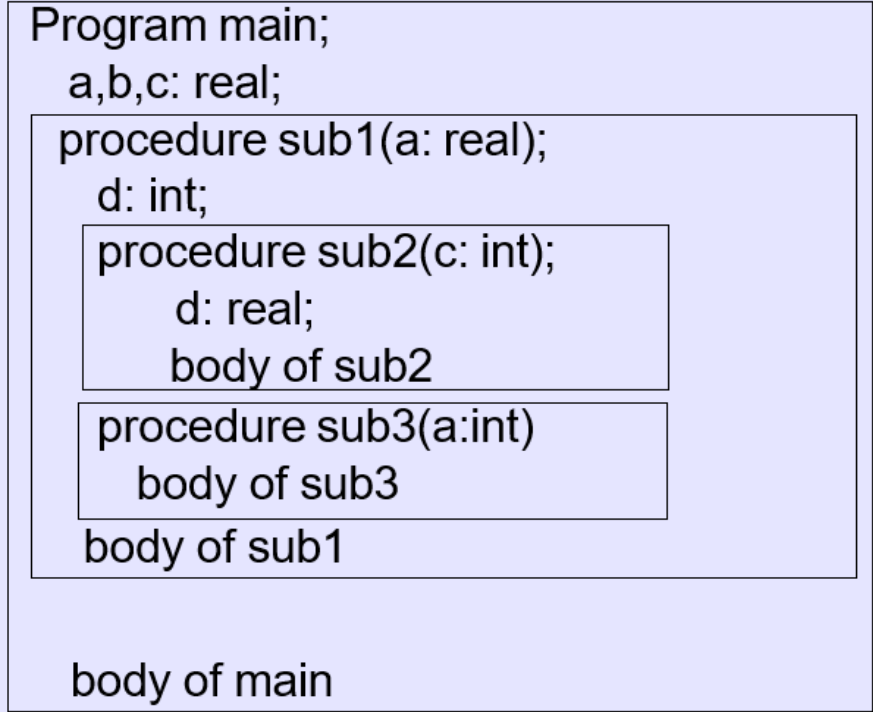
procedure sub3(a:int)

body of sub3

body of sub1

body of main

Procedure parameters shadow any variable of the same name in the enclosing scope.



| Procedure | Enclosing | Local:addr<br>= offset | Non-local: addr<br>=(scope,offset)                |
|-----------|-----------|------------------------|---------------------------------------------------|
| main      | -         | a:0,b:1,c:2            | -                                                 |
| sub1      | main      | a:0,d:1                | b:(main (1), 1), c:(main (1), 2)                  |
| sub2      | sub1      | c:0,d:1                | b:(main (2), 1), a:(sub1 (1), 0)                  |
| sub3      | sub1      | a:0                    | b:(main (2), 1), c:(main (2), 2)<br>d:(sub1(1),1) |

# Example Program

Program main;

```
procedure sub1(a: int,b:int);
```

```
  procedure sub2(c: int);
```

```
    if c > 0 call sub2(c-1)
```

```
  procedure sub3(b: int)
```

```
    body of sub3
```

```
call sub3(b); call sub2(a);
```

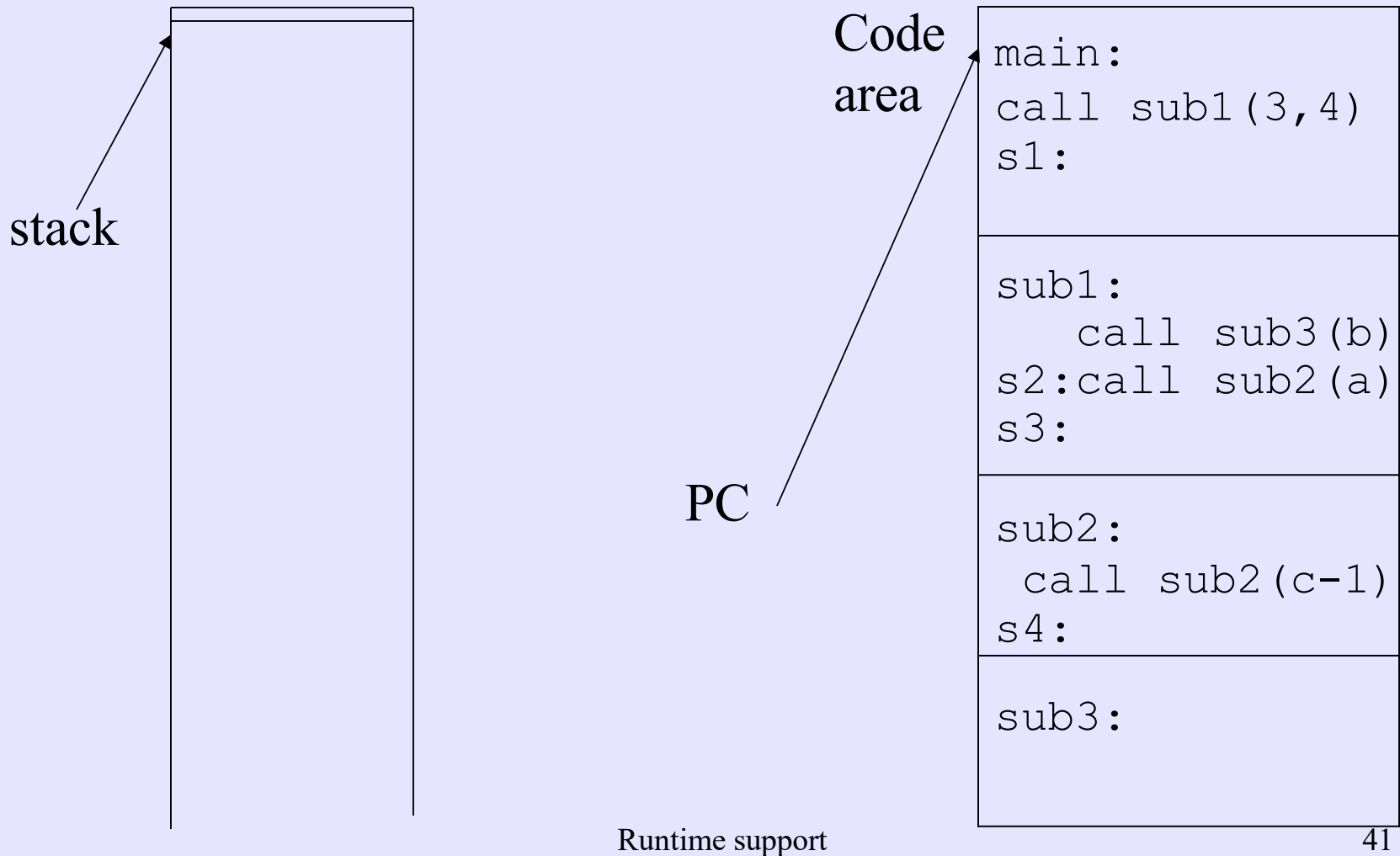
```
call sub1(3,4);
```

Similar to the previous one

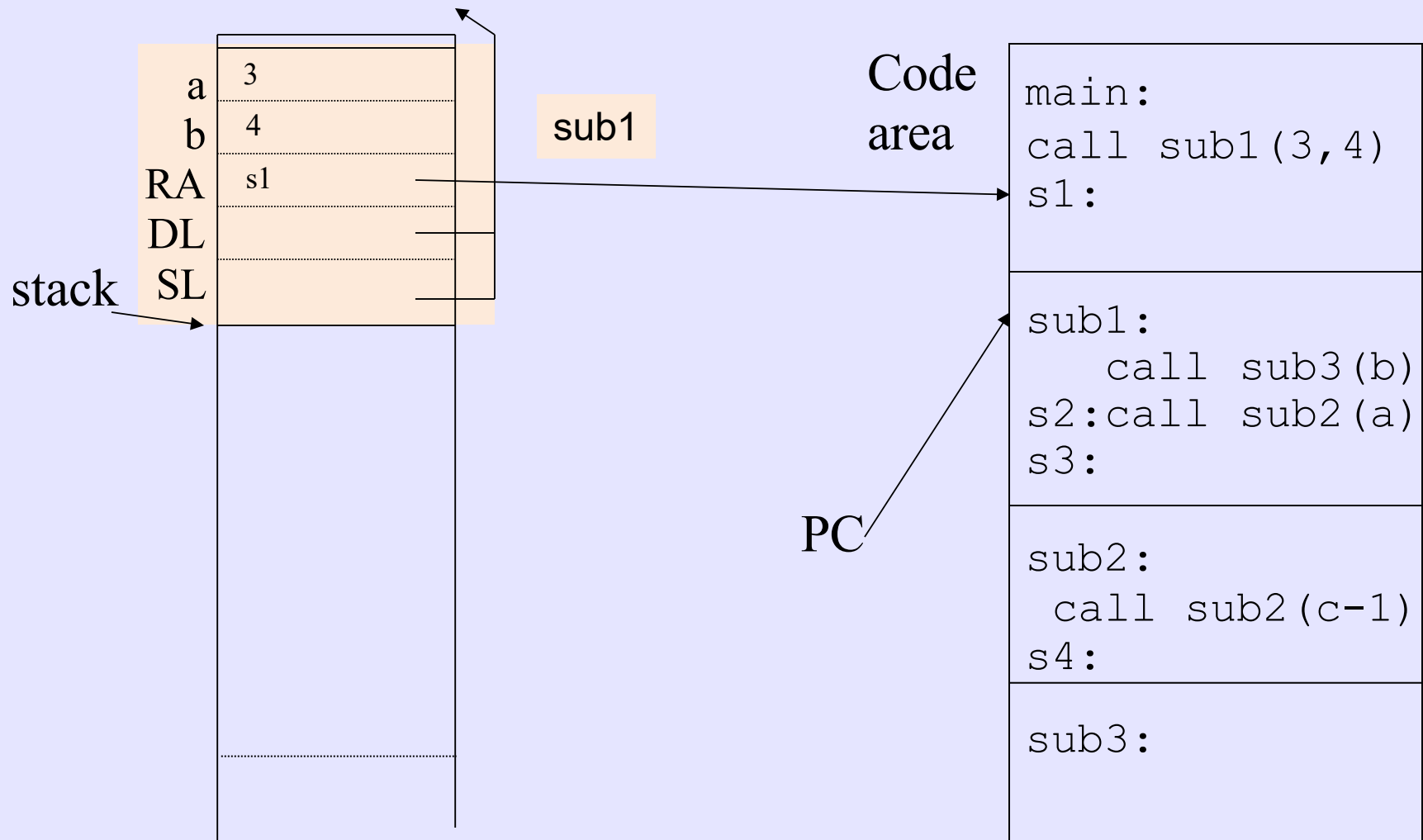
- Function definition nesting remains the same
- Changed - parameters names & count
- No local variables
- Added - bodies of main, sub1 and sub2



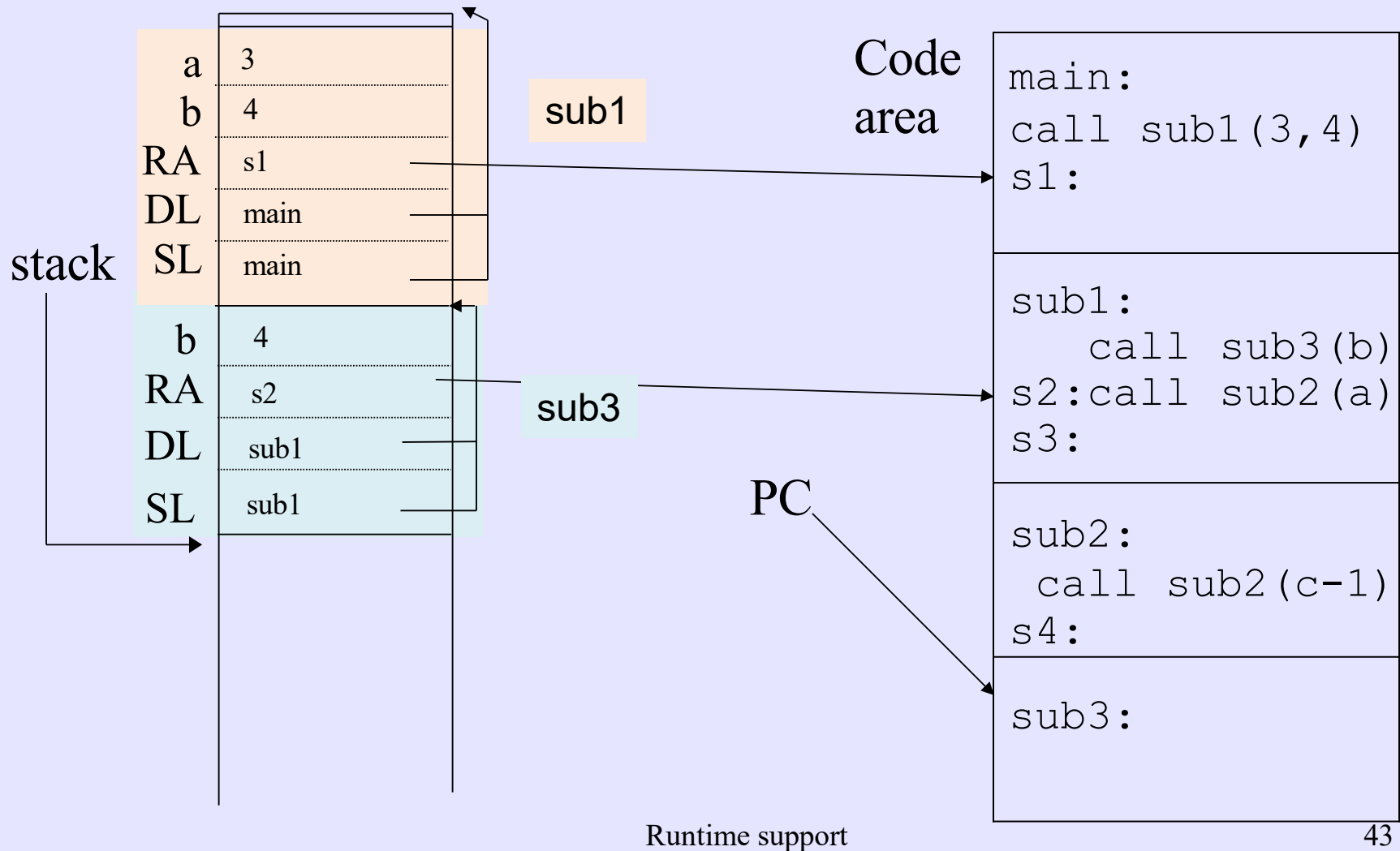
# Example Program at runtime 1



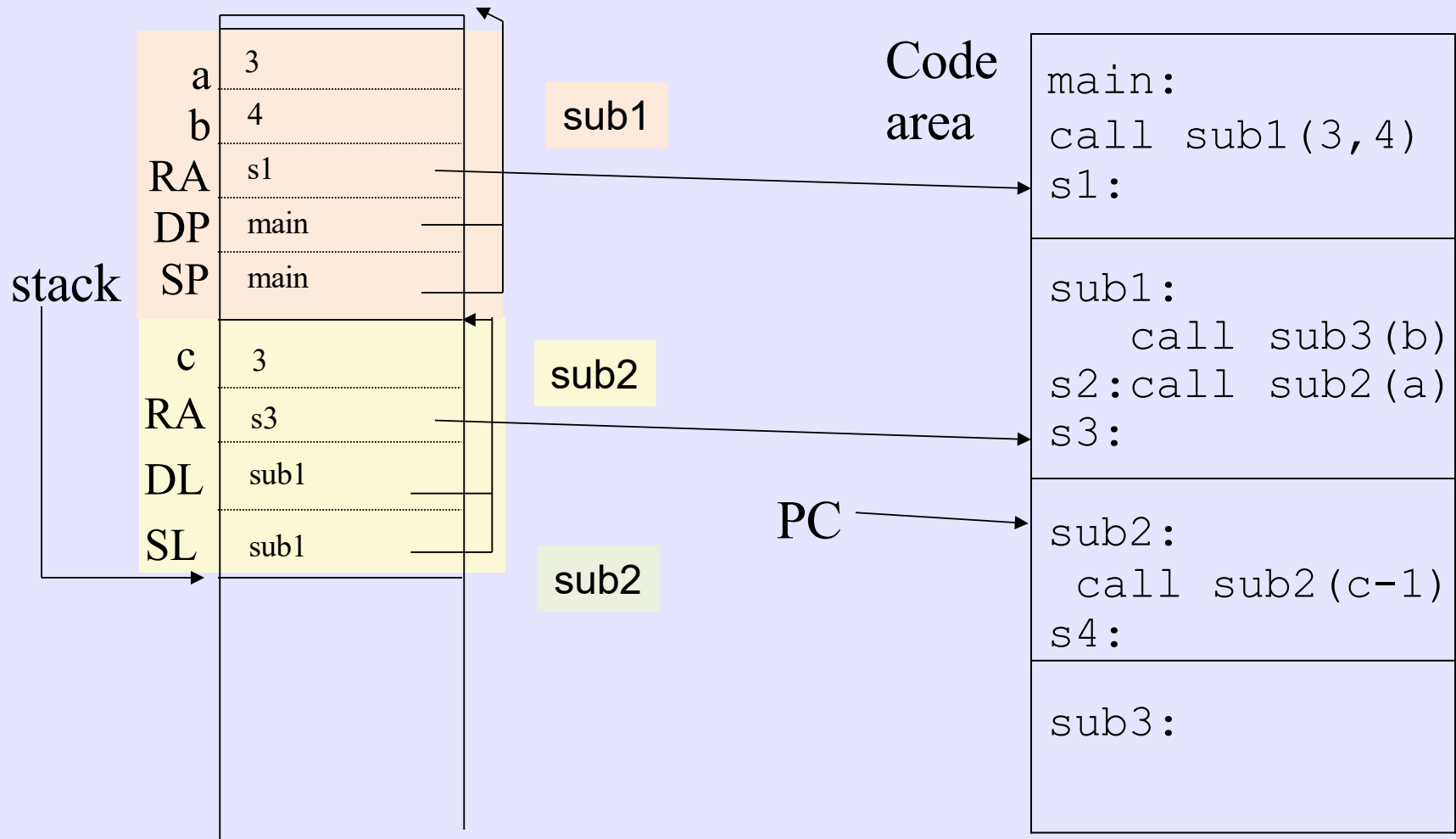
# Example Program at runtime 2



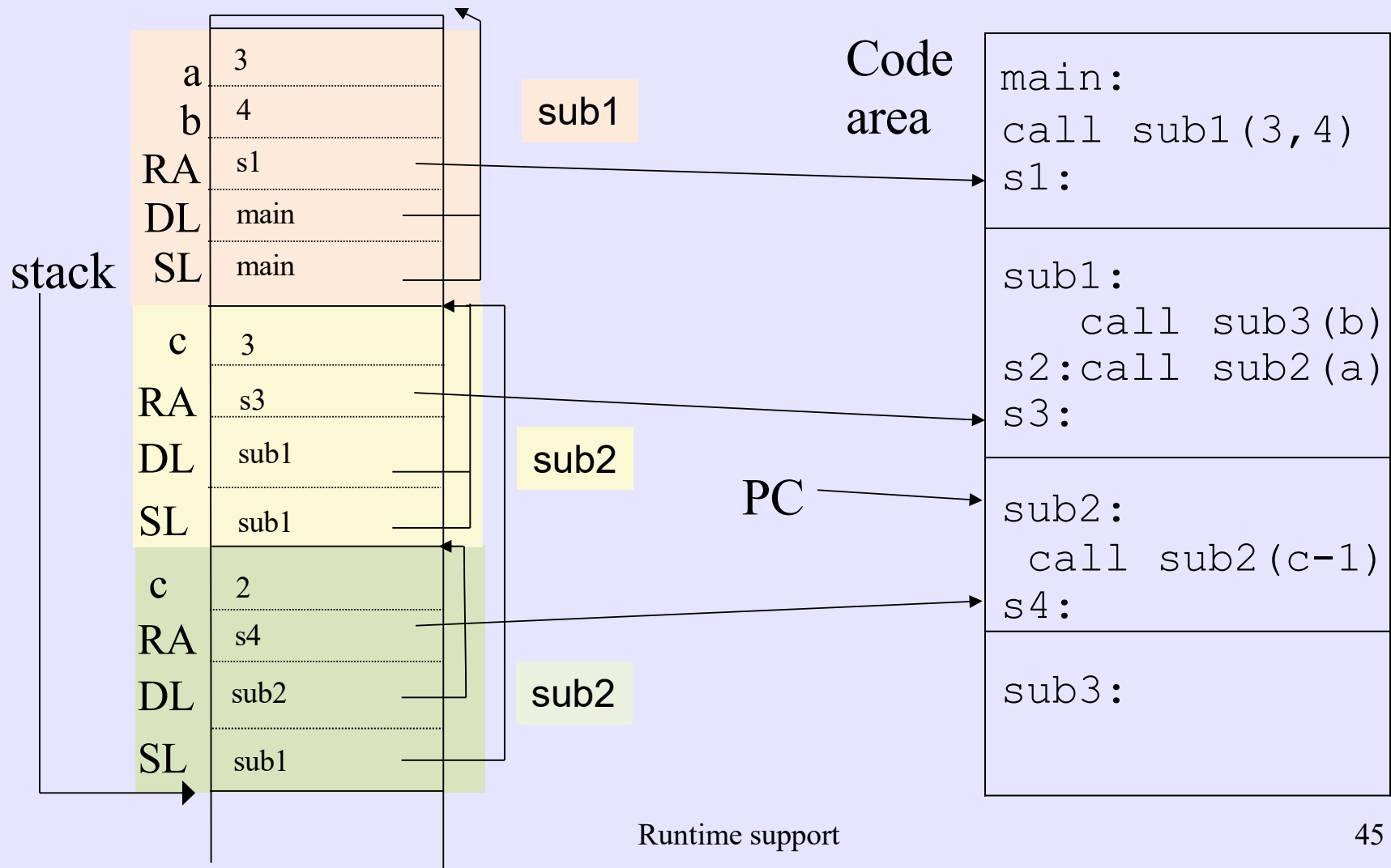
# Example Program at runtime 3



# Example Program at runtime 4



# Example Program at runtime 5



# Display

Alternate representation for access information.

- The current static chain kept in a single location
- Advantages: faster addressing
- Disadvantages: additional data structure to store and maintain.

# Display

## Array **d**

- contains one pointer for each nesting depth
- **d[i]** holds a pointer to the last (highest) activation record on the stack for any procedure at nesting depth **i**

If a procedure **p** is executing, and needs to access variable **x** belonging to some procedure **q**, we only need to follow the pointer in **d[i]** (and add offset for **x**), where **i** is the nesting depth of **q** (the compiler knows what **i** is)

# Example (Dragon Book)

```
1) fun sort(inputFile, outputFile) =  
    let  
2)        val a = array(11,0);  
3)        fun readArray(inputFile) = ... ;  
4)            ... a ... ;  
5)        fun exchange(i,j) =  
6)            ... a ... ;  
7)        fun quicksort(m,n) =  
            let  
8)                val v = ... ;  
9)                fun partition(y,z) =  
10)                    ... a ... v ... exchange ...  
                in  
11)                    ... a ... v ... partition ... quicksort  
                end  
            in  
12)                ... a ... readArray ... quicksort ...  
            end;
```

sort (s)  
    ↪ readArray (r)  
    ↪ exchange (e)  
    ↪ quicksort (q)  
        ↪ partition (p)

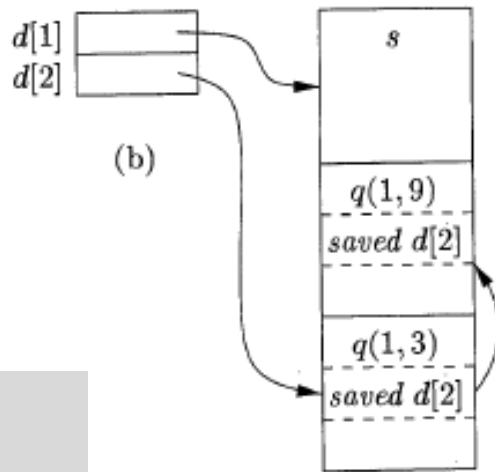
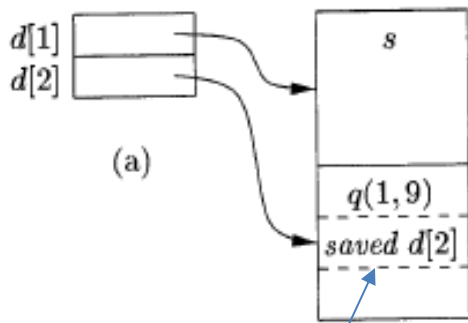
Nesting depths 1,2,3:

$n_s = 1$

$n_r = n_e = n_q = 2$

$n_p = 3$



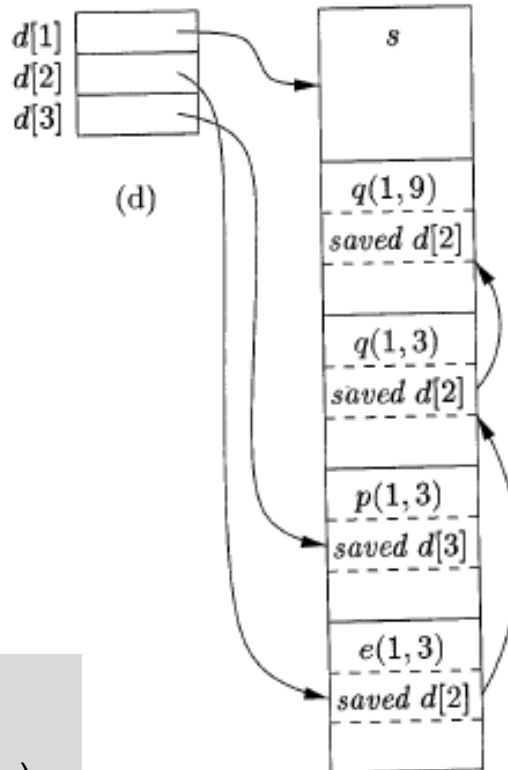
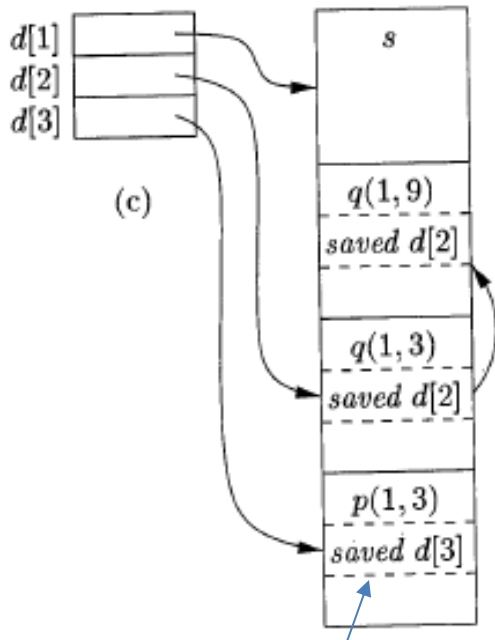


**call  $q(1,3)$ ,  $n_q = 2$**

Stack contains another AR  
for a procedure at depth 2 -  
 $q(1,9)$  call  
Old value of  $d[2]$  is saved

**call  $q(1,9)$ ,  $n_q = 2$**

Stack does not contain another AR for a  
procedure at depth 2, no save (null pointer)



**call  $e(1,3)$ ,  $n_e = 2$**

Stack contains another ARs  
for procedures at depth 2 -  
 $q(1,9)$  and  $q(1,3)$  calls  
Old value of  $d[2]$  is saved

**call  $p(1,3)$ ,  $n_p = 3$**

Stack does not contain another AR for a  
procedure at depth 3, no save (null pointer)

# Parameter Passing

- **Call-by-value** – data is copied at the callee and any item changes do not affect values in the caller.
- **Call-by-reference** – pointer to data is given to the callee and any changes made by the callee are indirect references to the actual value in the caller.
- **Call-by-value-result (copy-restore)** – hybrid of call-by-value and call-by-reference. Data copied at the callee. During the call, changes do not affect the actual parameter. After the call, the actual value is updated.
- **Call-by-name** – the actual parameter is in-line substituted into the called procedure. This means it is not evaluated until it is used.

# Call-by-value vs. Call-by-reference

```
var a,b : integer
```

```
procedure swap(x,y : integer);
```

```
  var t: integer;
```

```
  begin
```

```
    t := x; x := y; y := t;
```

```
  end;
```

```
begin
```

```
  a := 1; b := 2;
```

```
  swap(a,b); write ('a = ',a);
```

```
  write ('b = ',b);
```

```
end.
```

|          | value | reference |
|----------|-------|-----------|
| write(a) | 1     | 2         |
| write(b) | 2     | 1         |

# Call-by-value-result vs. Call-by-reference

```
var a: integer
procedure foo(x: integer);
  begin
    a := a + 1;
    x := x + 1;
  end;
begin
  a := 1;
  foo(a);
  write ('a =', a);
end.
```

|          | Value-result | reference |
|----------|--------------|-----------|
| write(a) | 2            | 3         |

# Call-by-name vs Call-by-reference

```
var a : array of integer; i: integer
procedure swap(x,y : integer);
  var t: integer;
  begin t := x; x := y; y := t; end;
  t := i; i := a[i]; a[i] := t;
begin
  i := 1;
  swap(i,a[i]);
  write ('i,a[i] = ',i,a[i]);
end.
```



If  $a[i]$  is initialized to  $i+1$ :  
 $a[0] = 1, a[1] = 2, a[2] = 3, \dots$

## call-by-name

```
i = 2
a[1] = 2 // unchanged
a[2] = 1
```

```
t := i;    // t = i = 1
i := a[i]; // i = a[1] = 2
a[i] := t; // a[2] = 1
```

## call-by-reference

```
i = 2
a[1] = 1
a[2] = 3 // unchanged
```

## Parameter passing conventions

Formal parameters are always lvalues. Actual parameters are rvalues that may or may not also be lvalues.

All languages insist on the actual parameters and formal parameters agreeing in number (at least if the function expects a fixed number) and in type (if the language has types).

However, different languages have different ideas of what passing a parameter *means*. These ideas are formalized in parameter passing conventions, of which the following three are the most popular:

|                      |                   |
|----------------------|-------------------|
| Call by value        | C, ML             |
| Call by value-result | Ada, some Fortran |
| Call by reference    | Pascal            |

## Argument evaluation order

When passing arguments on the stack, it is usually most convenient for callers to evaluate arguments right to left. That way, each evaluated actual parameter can be pushed on the stack, with the last pushed parameter being the first parameter. This parameter ends up on top, where the callee with variable-length argument lists can use it to figure out how many other arguments there are.

When passing arguments in registers, it is usually most convenient for callers to evaluate arguments left to right. That way, when evaluating argument  $N$  into (say) register  $N$ , all the registers with numbers higher than  $N$  are available for subexpressions.

To leave themselves freedom to pick either of these approaches, language designers usually explicitly leave the argument evaluation order undefined.

# Adresovanie polí

- Po riadkoch
- Po stĺpcoch
- Od prvého elementu
- Od fiktívnej adresy  $A[0, \dots, 0]$
- Oba spôsoby sa dajú efektívne optimalizovať



## Arrays

month: array[1..12] of integer

var: array[lo..hi] of type

The amount of memory required by the array is the number of elements ( $hi - lo + 1$ ) multiplied by the size of the element type (*eltsize*).

If the memory allocated to *var* starts at *start*, then the address of *var*[*i*] is  $start + (i - lo) * eltsize$ .

Some languages, such as C, require *lo* to be zero to avoid the subtraction. The multiplication is usually done using shifts, even if *eltsize* is not a power of 2. For example, multiplication by 12 can be done using two shifts and an add:

$$x * 12 == x * 8 + x * 4 == x \ll 3 + x \ll 2$$

Strength  
reduction

# Polia (arrays)

Polia ukladáme súvisle prvok po prvku:  $A[\text{low}:\text{high}]$  nech  $w$  je veľkosť jedného prvku. Potom adresa  $i$ -tého prvku je:

$$\text{base} + (i - \text{low}) * w = \text{base} - \text{low} * w + i * w = A[0] + i * w$$

Pre dvojrozmerné polia, adresa prvku  $A[i_1, i_2]$  je

$$A[0,0] + i_1 * w_1 + i_2 * w_2$$

Alternatívne ak  $n_2$  je počet prvkov druhej dimenzie a  $w = w_2$  (šírka typu prvkov poľa)

$$\begin{aligned} & A[0,0] + i_1 * (w * n_2) + i_2 * w = \\ & = A[0,0] + (i_1 * n_2 + i_2) * w \end{aligned}$$

## Multidimensional arrays

N-dimensional arrays can be handled as arrays whose elements are themselves (N-1)-dimensional arrays.

*var*: array[*lo1*..*hi2*, *lo2*..*hi2*, *lo3*..*hi3*] of type

The address of *var*[*i*, *j*, *k*] is

*start*

$$\begin{aligned} &+ (i - lo1) * ((hi2 - lo2 + 1) * (hi3 - lo3 + 1) * eltsize) \\ &+ (j - lo2) * ((hi3 - lo3 + 1) * eltsize) \\ &+ (k - lo3) * (eltsize) \end{aligned}$$

This can be more easily computed as

$$start + (((((i - lo1) * s2) + (j - lo2)) * s3) + (k - lo3)) * eltsize$$

using the constants  $s2 = hi2 - lo2 + 1$  and  $s3 = hi3 - lo3 + 1$ .

# Structures

The layout of structures is also governed by alignment considerations, since most languages don't allow fields to be reordered. In the structure below, bytes 5-7 and 12-15 are padding.

The alignment requirement on a structure is the most severe alignment requirement of any of its fields, so the structure below must have an address divisible by 8. Even though one structure needs only 25 bytes, arrays of it need 32 bytes per element.

```
/* size    bytes */
typedef struct {
    int    f1;    /* 4    0-3    */
    char    f2;    /* 1     4     */
    int    f3;    /* 4    8-11   */
    double  f4;    /* 8   16-23  */
    char    f5;    /* 1    24     */
} padded;
```

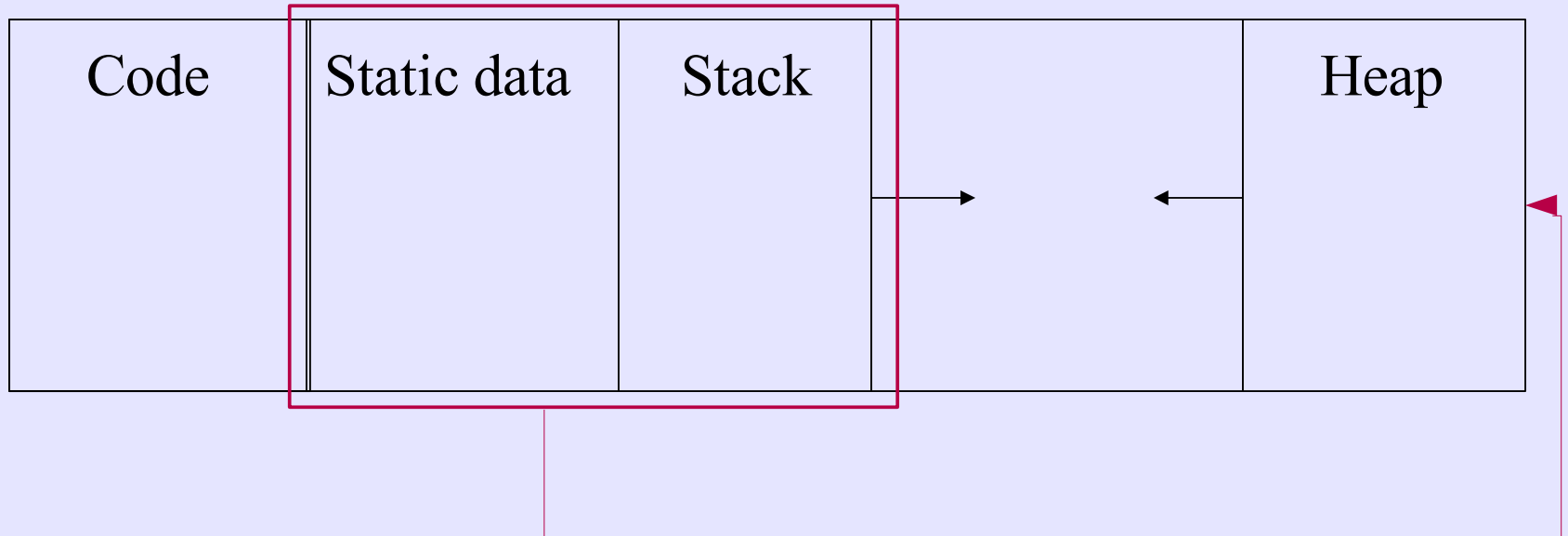
Typ musí byť na  
adrese deliteľnej  
jeho veľkosťou

# Heap Allocation

Dynamic allocation may be explicit or implicit in the language.

- How can we keep track of what areas are free?
- How can we prevent fragmentation?
- Garbage -- storage location that becomes unreachable
- Dangling reference -- pointer to a memory location that is free
- Manual heap management vs automatic garbage collection

# Storage Organization



+ registers

# Garbage Collection

Garbage collection is the process of automatically locating and reclaiming unused memory.

- Three major classes of garbage collectors: mark-sweep, generational, reference count.
- A collector that requires the program to halt during the collection is a stop/start collector; else it is a concurrent collector.
- Garbage collection is a big deal in functional or logic languages which use a lot of dynamic data.
- **Avoid if possible!**

# Reference counting

- Pre každý objekt sa sleduje počet referencií, ktoré naň odkazujú
- Ak sa počet referencií rovná nule, objekt je nedostupný a môže sa odstrániť
- Konceptne jednoduchší spôsob na implementáciu
- Má niektoré obmedzenia (napr. riešenie cyklických referencií)
- Často **integrovaný v kompilátore** s dodatočnou podporou počas behu
  - Kompilátor generuje kód, ktorý obsahuje aj operácie na zvýšenie / zníženie počtu referencií na objekty na vhodných miestach programu
  - Počítanie referencií je zahrnuté priamo v spustiteľnom kóde vytvorenom kompilátorom



# Tracing GC – Mark & sweep

## **Sledovacie algortimy** (tracing GC)

- Najbežnejší spôsob (napr. Garbage-first pre Javu - G1 GC)
- Proces na pozadí, ktorý dealokuje objekty asynchrónne počas behu program
- **Mark & sweep algoritmus** - prehľadáva sa graf objektov
  - Začína sa v koreňoch grafu = živé referencie (registre, call stack, glob. premenné, konštanty, ..)
  - Prechádza sa graf, každý dosiahnutý objekt sa označí
  - Neoznačené objekty sa môžu odstrániť
  - (voliteľné) Skompaktnenie pamäte

# Generational GC

Heap je rozdelený na viacero generácií (2-3)

„Mladé“ generácie sa prechádzajú často (keď prežijú niekoľko scanov, presunú sa do staršej generácie) – malé, rýchle

„Staré“ generácie sa prechádzajú menej často  
- väčšie, pomalé

Založené na predpoklade, že väčšina objektov má krátku životnosť

# Tabuľka symbolov

- Jediný dôvod udržiavať ju aj počas behu je ladenie (debugging).
- Má byť malá a kompaktná. Doporučuje sa informáciu premenlivej dĺžky (identifikátory, konštanty) udržiavať v heape a v tabuľke symbolov len smerníky.
- Z dôvodov jednoduchosti je najvhodnejšie jednoduché hašovanie alebo lineárne vyhľadávanie.
- Ak jazyk umožňuje „nested scopes“ (vnorené bloky) musí to podporovať aj tabuľka symbolov.

# Symbol table structure

Each symbol table stores

- a pointer to the symbol table of the enclosing scope (if any)
- pointers to the symbol tables of the scopes it encloses
- whether it has its own stack frame (e.g. blocks don't)
- information about the named entities (such as types, variables, functions) declared in its scope.

The information associated with each name depends on what it is declared to be; you need to record different information for e.g. functions than for variables.

In C, only the global scope may contain function definitions. Other languages don't have this restriction.

## Symbol tables

The symbol table for a scope should map each name declared in the scope to all the information the compiler has about that name. This includes information given in the declaration (e.g. type) as well as information computed by the compiler: size, alignment requirement, and address, as offset from the frame pointer or the start of this module's rodata, data or bss section.

The counters you need for memory allocation are logically also part of the symbol table.

The structure of the symbol table should allow quick access not only when the table is small (which is typical for human-written code) but also when it is very big (which can happen when the source code itself is created automatically, by tools such as lex and yacc).

This usually requires a balanced tree or an expandable hash table.

Mala by byť preferovaná separátna kompilácia. Program sa má skladať z relatívne malých nezávislých blokov.

## Handling nested scopes

In most languages, scopes can nest inside each other. For example, C has three kinds of scopes: global, function and block, and they all nest.

The compiler needs to keep a separate symbol table for each scope. Each of these symbol tables maps each name declared in the corresponding scope to information about that declaration.

The symbol tables themselves form a tree based on the nesting. At any point in the code, there is a current symbol table, the symbol table for the current scope.

Looking up a name requires looking in the current symbol table, and if not found, searching its ancestors in turn.

A declaration in an inside scope can hide a declaration in an outer scope.